

INFORME DEL PROYECTO
ÁRBOL BINARIO DE BÚSQUEDA (BST)

Universidad Da Vinci de Guatemala

Facultad de Ingeniería en Sistemas y Ciencias de la Computación

Curso: Estructuras de Datos

Proyecto: Implementación de Árbol Binario de Búsqueda (BST)

Lenguaje utilizado: Java

Estudiante: Gustavo Rafael Juarez Mendoza

Carnet: 202505481

1. Introducción

Las estructuras arbóreas son fundamentales en el área de programación y ciencias de la computación, ya que permiten organizar información de manera jerárquica y eficiente. En este proyecto se implementó un Árbol Binario de Búsqueda (BST), el cual es una estructura de datos que almacena elementos siguiendo reglas específicas de ordenamiento. El objetivo principal del proyecto fue comprender el funcionamiento interno de esta estructura y aplicar sus operaciones principales utilizando el lenguaje Java.

2. Objetivos

Objetivo General:

Desarrollar un sistema en Java que implemente un Árbol Binario de Búsqueda capaz de insertar, buscar, eliminar y recorrer nodos.

Objetivos Específicos:

- Comprender el funcionamiento de los árboles binarios.
- Implementar métodos de inserción, búsqueda y eliminación.
- Analizar la complejidad algorítmica de las operaciones.
- Aplicar recorridos en profundidad del árbol.
- Explicar el funcionamiento práctico de la estructura.

3. ¿Qué es un Árbol Binario de Búsqueda?

Un Árbol Binario de Búsqueda es una estructura de datos compuesta por nodos. Cada nodo contiene un valor y referencias hacia dos posibles hijos: hijo izquierdo e hijo derecho.

La regla principal de un BST es:

- Los valores menores al nodo actual se colocan a la izquierda.
- Los valores mayores se colocan a la derecha.

Gracias a esta organización, las búsquedas pueden realizarse de manera más rápida comparadas con estructuras lineales.

4. Estructura del Nodo

Cada nodo del árbol contiene:

- Un dato entero.
- Una referencia al hijo izquierdo.
- Una referencia al hijo derecho.

La implementación en Java permite conectar nodos entre sí para formar la estructura jerárquica del árbol.

5. Operaciones Implementadas

El proyecto implementa las operaciones más importantes de un BST:

1. Inserción:

Permite agregar nuevos valores respetando las reglas del árbol.

2. Búsqueda:

Permite verificar si un valor existe dentro del árbol.

3. Eliminación:

Elimina nodos considerando tres casos diferentes:

- Nodo sin hijos.
- Nodo con un hijo.
- Nodo con dos hijos.

4. Recorridos:

- InOrder
- PreOrder
- PostOrder

6. Explicación del Método Insert

El método insert se encarga de agregar un nuevo valor al árbol. El algoritmo compara el nuevo dato con el nodo actual. Si el valor es menor, se mueve hacia la izquierda; si es mayor, se mueve hacia la derecha.

Este proceso continúa hasta encontrar una posición vacía donde el nuevo nodo será agregado.

7. Explicación del Método Search

El método search permite encontrar un valor dentro del árbol. El algoritmo compara el valor buscado con el nodo actual y decide si continuar hacia la izquierda o hacia la derecha.

Si el valor es encontrado, el método devuelve verdadero; si no existe, devuelve falso.

8. Explicación del Método Delete

La eliminación es una de las operaciones más complejas del BST.

Caso 1: Nodo hoja

Si el nodo no tiene hijos, simplemente se elimina.

Caso 2: Nodo con un hijo

El hijo reemplaza la posición del nodo eliminado.

Caso 3: Nodo con dos hijos

Se busca el menor valor del subárbol derecho para reemplazar el nodo eliminado.

9. Recorridos del Árbol

Los recorridos permiten visitar todos los nodos del árbol.

- InOrder:

Visita izquierda, raíz y derecha. Este recorrido muestra los valores ordenados.

- PreOrder:

Visita raíz, izquierda y derecha.

- PostOrder:

Visita izquierda, derecha y raíz.

10. Complejidad Algorítmica

La eficiencia del BST depende de la forma del árbol.

Complejidad promedio:

- Inserción: $O(\log n)$
- Búsqueda: $O(\log n)$
- Eliminación: $O(\log n)$

Peor caso:

- Inserción: $O(n)$
- Búsqueda: $O(n)$
- Eliminación: $O(n)$

El peor caso ocurre cuando el árbol se desbalancea y toma forma similar a una lista enlazada.

11. Ventajas del BST

- Organización eficiente de datos.
- Búsquedas rápidas.
- Inserciones dinámicas.
- Fácil recorrido de información.
- Implementación sencilla.

12. Desventajas del BST

- Puede desbalancearse fácilmente.
- El rendimiento disminuye en el peor caso.
- Requiere memoria dinámica para los nodos.

13. Aplicaciones en la Industria

Los Árboles Binarios de Búsqueda son utilizados en:

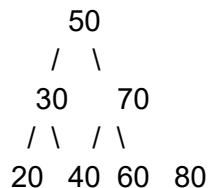
- Sistemas de bases de datos.
- Motores de búsqueda.
- Sistemas de archivos.
- Índices de información.
- Aplicaciones de organización de datos.

14. Ejemplo Práctico

Valores insertados:

50, 30, 70, 20, 40, 60, 80

Representación:



15. Conclusiones

El proyecto permitió comprender el funcionamiento de los Árboles Binarios de Búsqueda y la importancia de las estructuras jerárquicas en programación.

La implementación en Java ayudó a desarrollar habilidades relacionadas con estructuras dinámicas, recursividad y análisis de algoritmos.

Además, se comprobó que el BST es una estructura eficiente para almacenar y buscar información.

16. Recomendaciones

- Utilizar árboles balanceados para mejorar el rendimiento.
- Practicar diferentes recorridos del árbol.
- Comprender la lógica recursiva utilizada en las operaciones.
- Implementar más tipos de árboles en futuros proyectos.